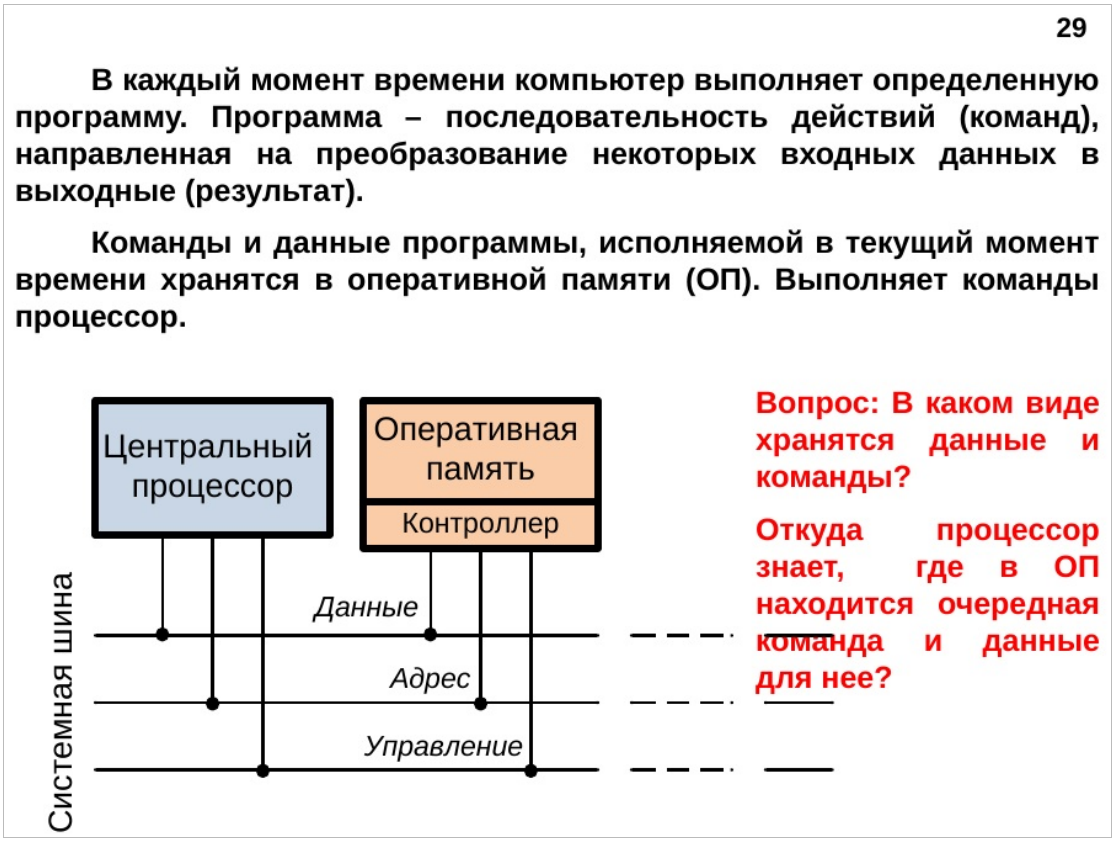
Конспект: процессор и рабочий цикл выполнения команды

Конспект по двум презентациям: «Взаимодействие устройств ЭВМ в процессе выполнения машинной команды (рабочий цикл процессора)» и лекции «Процессор» (элементы процессора, устройство управления, архитектуры системы команд, i8086, подпрограммы).

Часть 1. Рабочий цикл процессора

1.1. Данные и команды в памяти

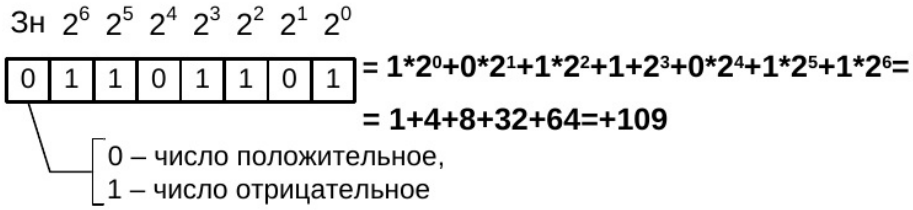
В каждый момент компьютер выполняет программу: последовательность команд, которая преобразует входные данные в результат. Команды и данные текущей программы лежат в **оперативной памяти (ОП)**, а выполняет их **процессор**.



Всё хранится в двоичном виде:

- числа: двоичная система, форматы со знаком, с фиксированной и плавающей точкой;
- символы: двоичные коды;
- двоичные векторы (для поразрядных логических операций): последовательности битов, обычно беззнаковые целые.

Представление данных
(подробно рассматривали в теме АЛУ)



Числа представляются в ЭВМ в двоичной системе счисления (форматы со знаком с фиксированной и с плавающей точкой).

Символы представляются их двоичными кодами.

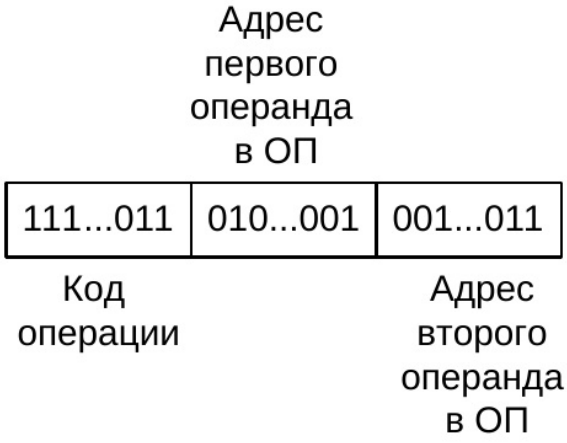
Двоичные вектора (над которыми производятся поразрядные логические операции) представляются последовательностями битов, для хранения которых, как правило, используются беззнаковые целые типы.

Команда — приказ компьютеру выполнить операцию. Двухадресная команда состоит из кода операции (**КОП**) и адресов двух операндов (**А1**, **А2**). Результат, как правило, записывается на место первого операнда.

Представление команд

Команда – это приказ компьютеру на выполнение какой-либо операции, например, операции сложения двух чисел (операндов), которые хранятся в оперативной памяти.

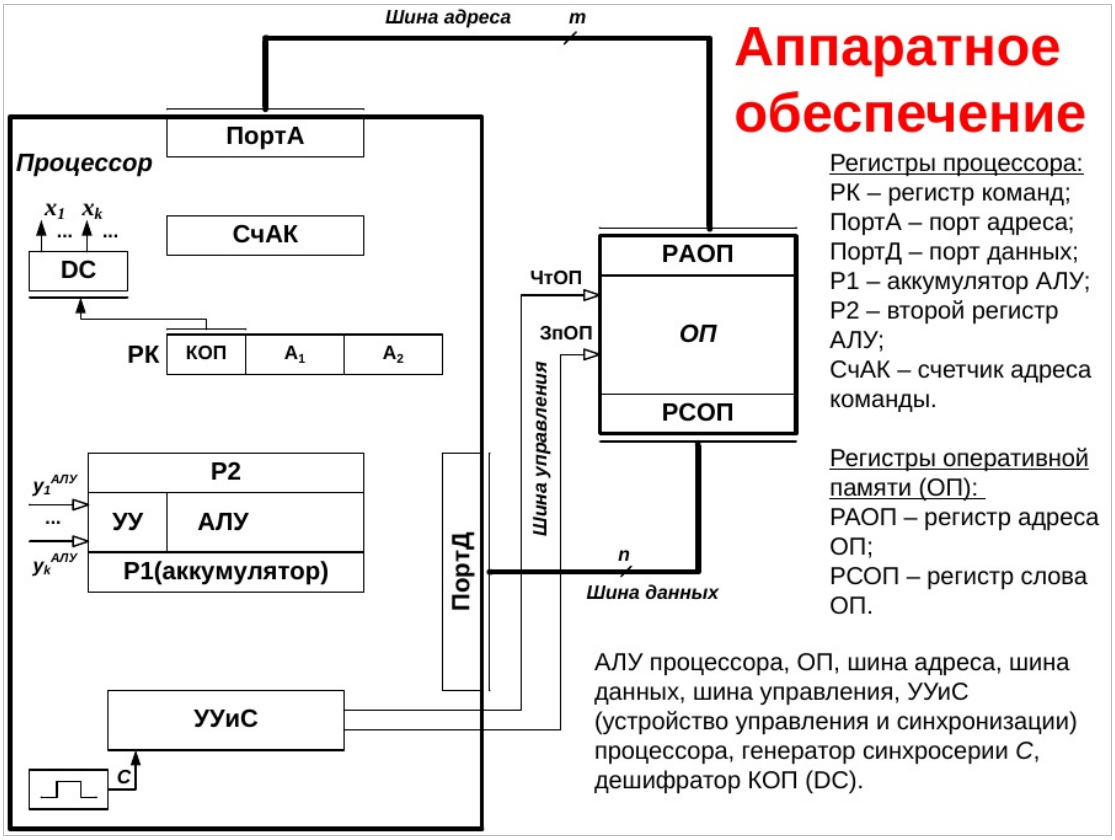
Можно представить команду в следующем формате:



Результат, как правило, помещается в память на место первого операнда

Набор команд конкретного процессора называется **системой команд**. В неё входят арифметические и логические команды, пересылки между регистрами и ОП, команды передачи управления, ввода-вывода.

1.2. Устройства и регистры



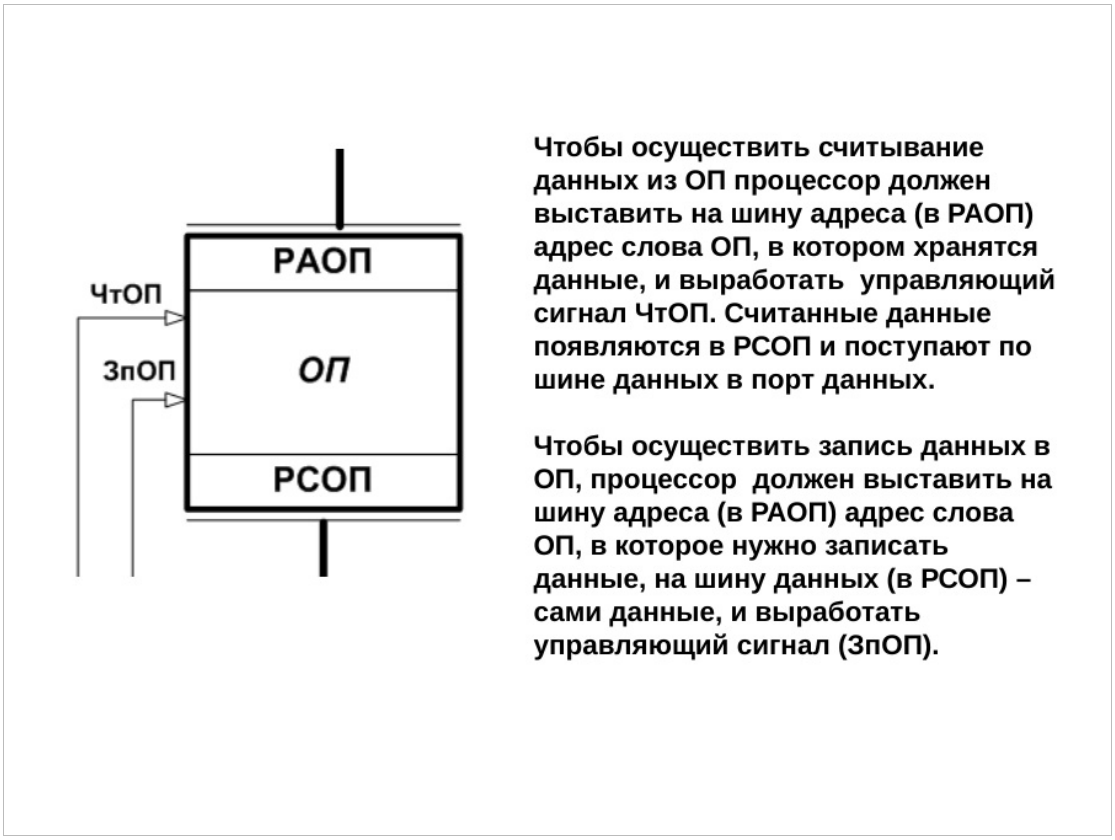
Регистры процессора

Обозначение	Название	Назначение
РК	регистр команд	хранит выбранную команду на всё время её выполнения (длина = макс. длине команды)
СчАК (СК, РС)	счётчик адреса команды	адрес очередной команды (длина зависит от объёма ОП)
ПортА	порт адреса	подключён к шине адреса
ПортД	порт данных	подключён к шине данных
Р1	аккумулятор АЛУ	первый операнд, затем результат
Р2	второй регистр АЛУ	второй операнд

Регистры ОП: РАОП (регистр адреса) и РСОП (регистр слова).

Устройства: АЛУ, УУиС (устройство управления и синхронизации, управляет всеми остальными), генератор синхросерии, дешифратор КОП (DC), шины адреса, данных и управления.

1.3. Чтение и запись ОП



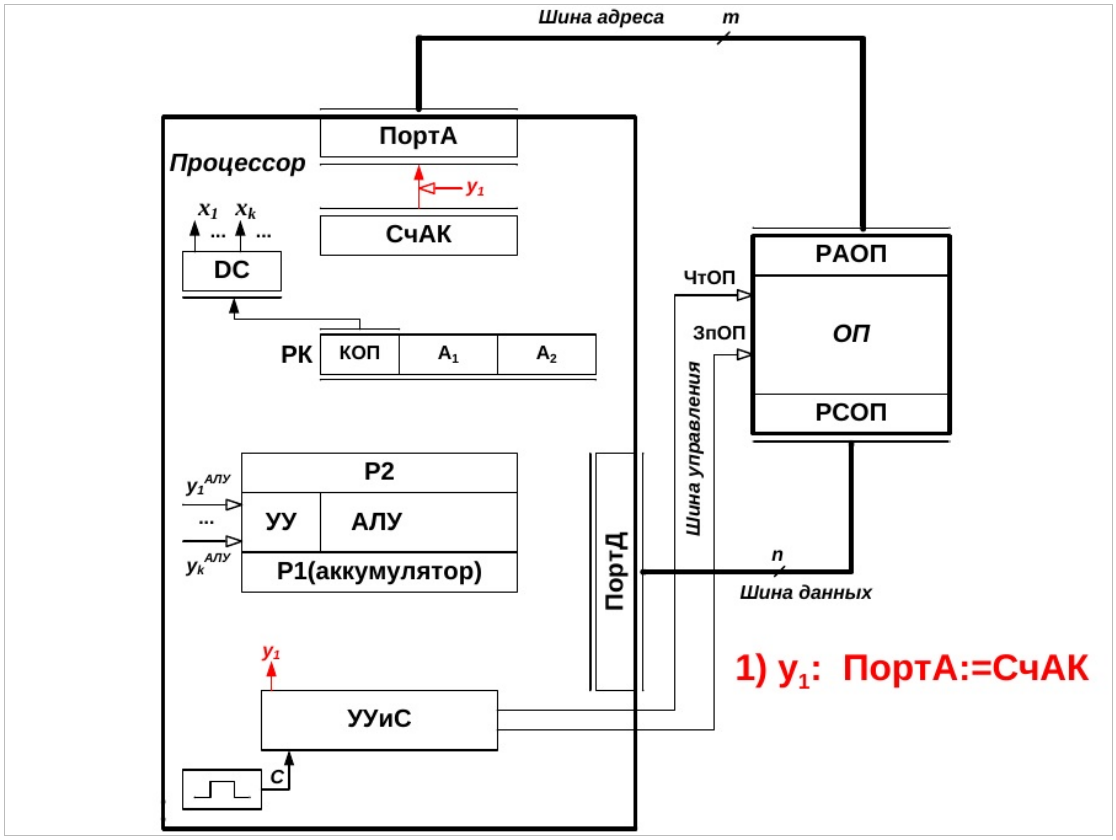
- **Чтение.** Процессор выставляет адрес слова на шину адреса (в РАОП) и вырабатывает сигнал **ЧтОП**. Данные появляются в РСОП и по шине данных приходят в ПортД.
- **Запись.** Процессор выставляет адрес на шину адреса (в РАОП), данные на шину данных (в РСОП) и вырабатывает сигнал **ЗпОП**.

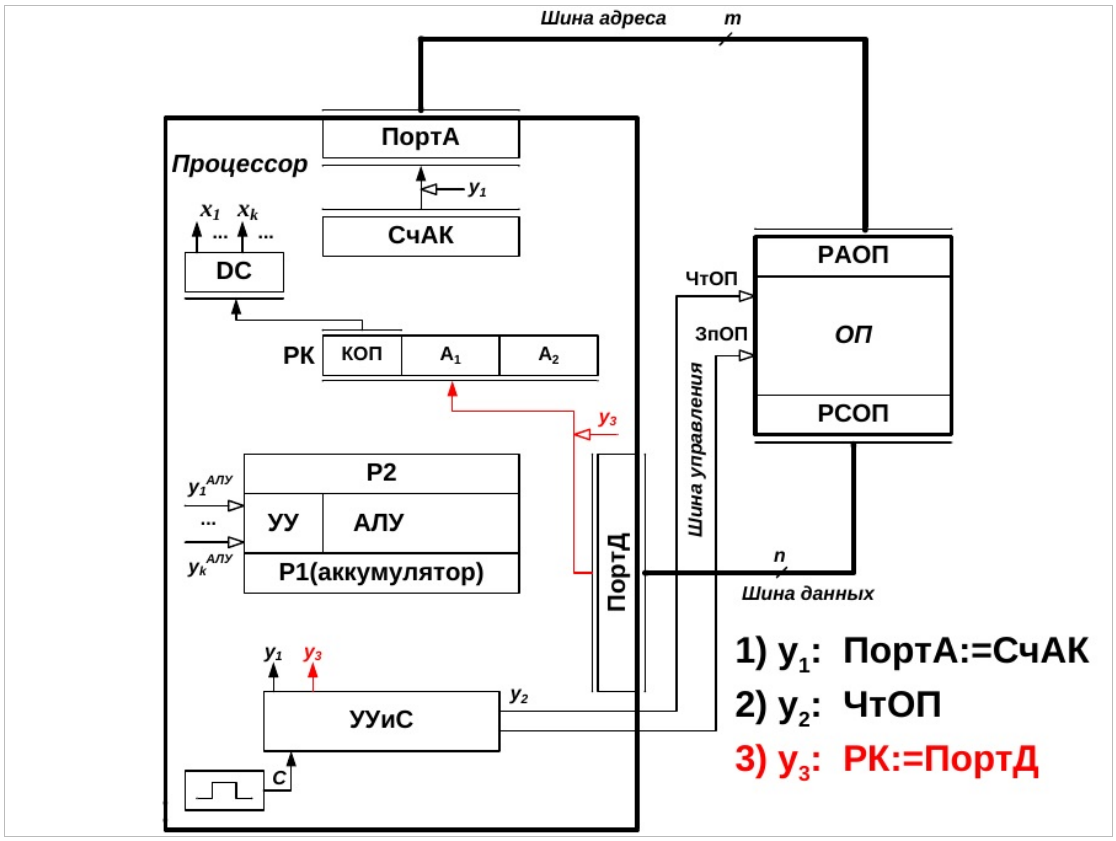
1.4. Пять этапов рабочего цикла

Рассматривается упрощённый вариант на примере двухадресной арифметической (или логической) команды. Каждая микрооперация — это управляющий сигнал $\mathbf{y_i}$, который вырабатывает УУиС.

Этап 1. Выборка команды из ОП

1. y1: `ПортА := СчАК`
2. y2: `ЧтОП`
3. y3: `РК := ПортД`

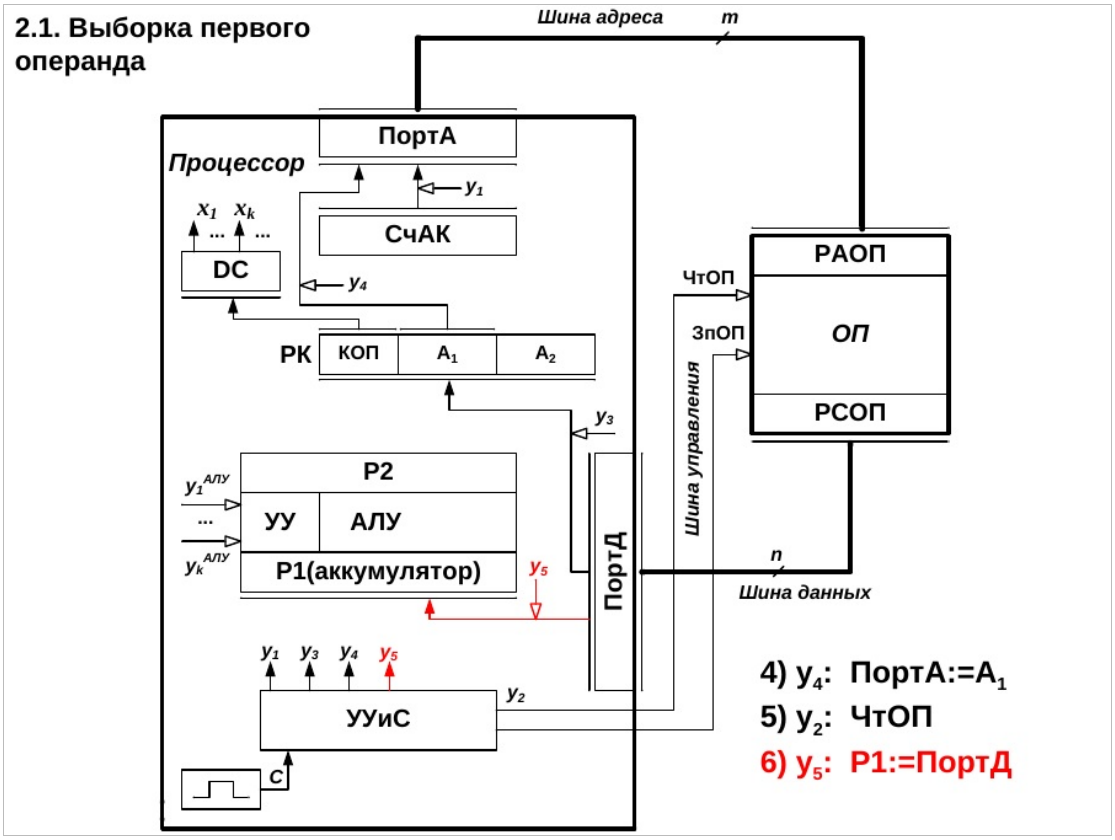




Этап 2. Выборка операндов из ОП

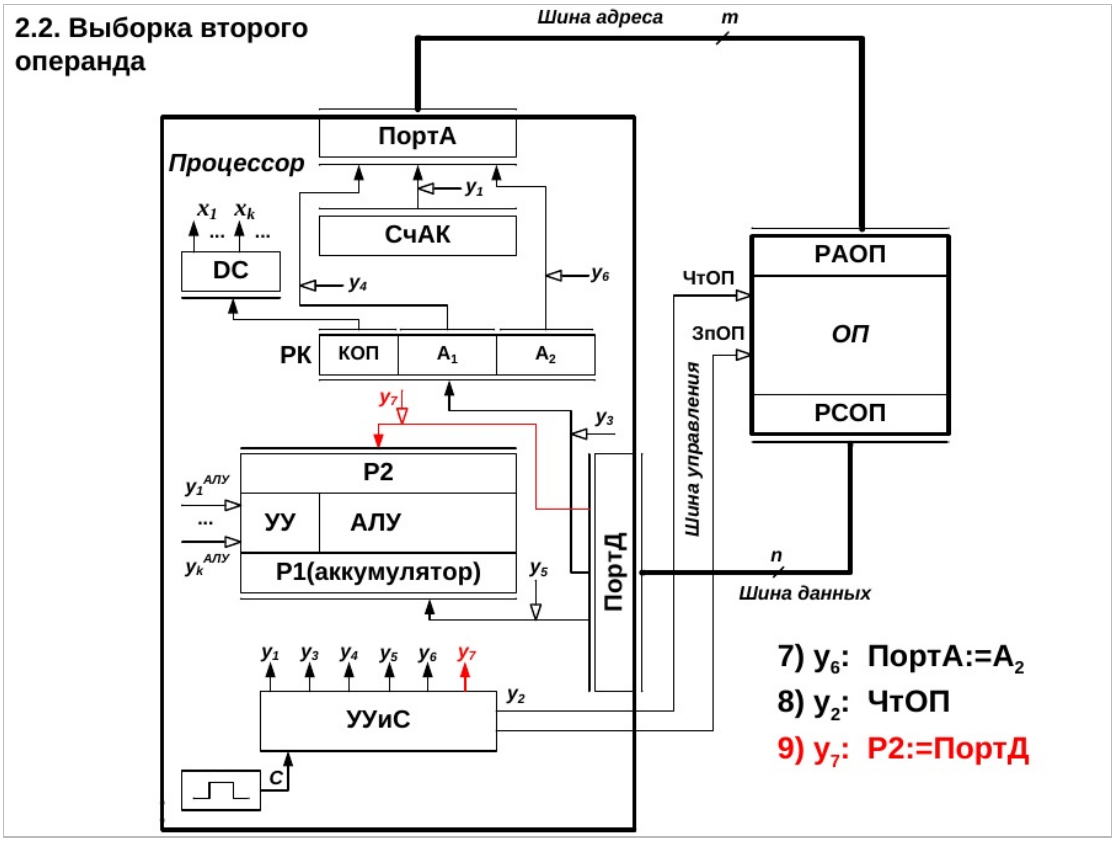
Первый операнд:

1. y4: `ПортА := A1`
2. y2: `ЧтОП`
3. y5: `Р1 := ПортД`



Второй операнд:

1. y6: `ПортА := A2`
2. y2: `ЧтОП`
3. y7: `Р2 := ПортД`

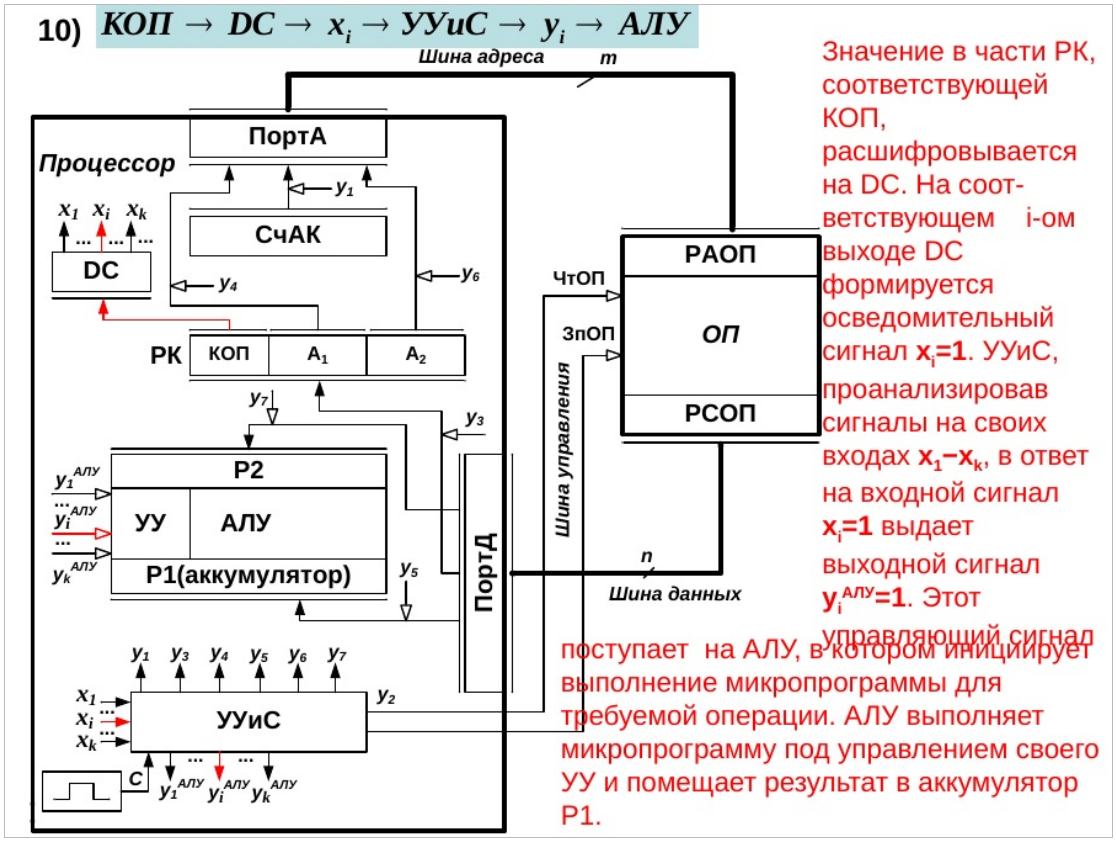


Этап 3. Выполнение операции в АЛУ

1. Код операции из РК расшифровывается на DC: на i-м выходе появляется осведомительный сигнал x_i = 1. УУиС в ответ выдаёт управляющий сигнал y_i^АЛУ = 1. АЛУ запускает микропрограмму нужной

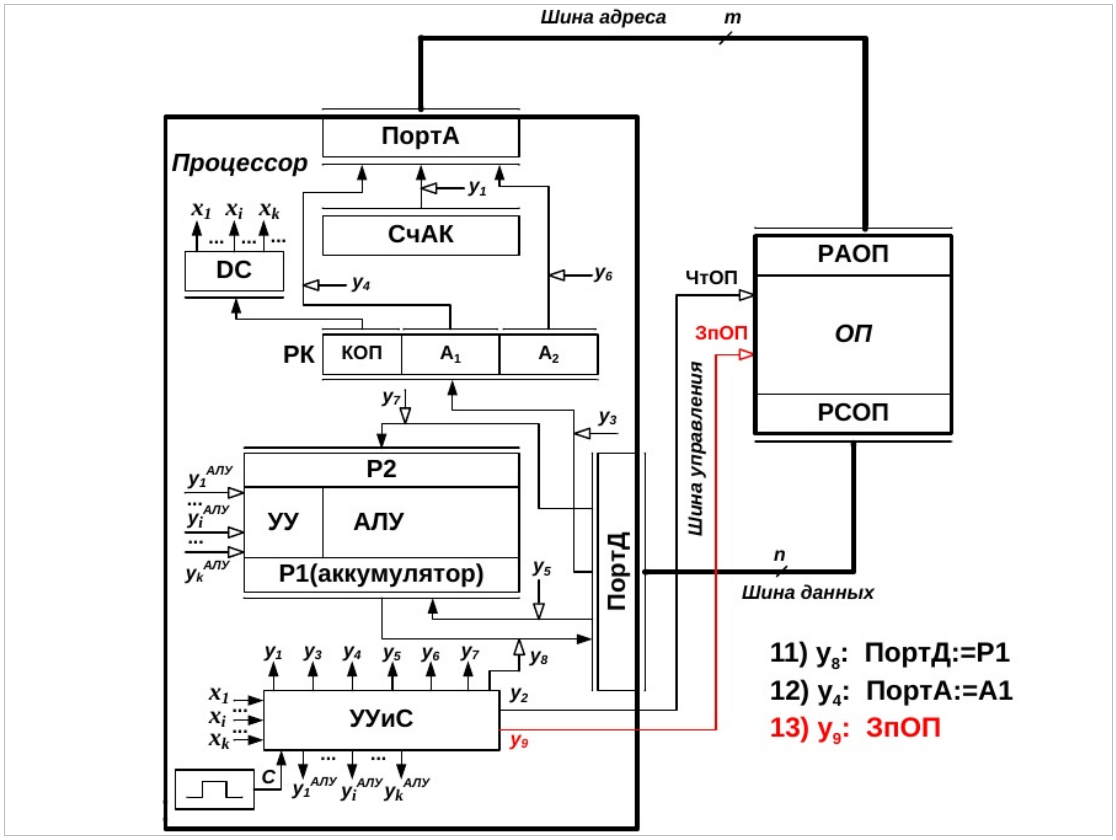
операции и помещает результат в аккумулятор Р1.

Цепочка сигналов: **КОП → DC → x_i → УУиС → y_i → АЛУ**.



Этап 4. Запись результата в ОП (на место первого операнда)

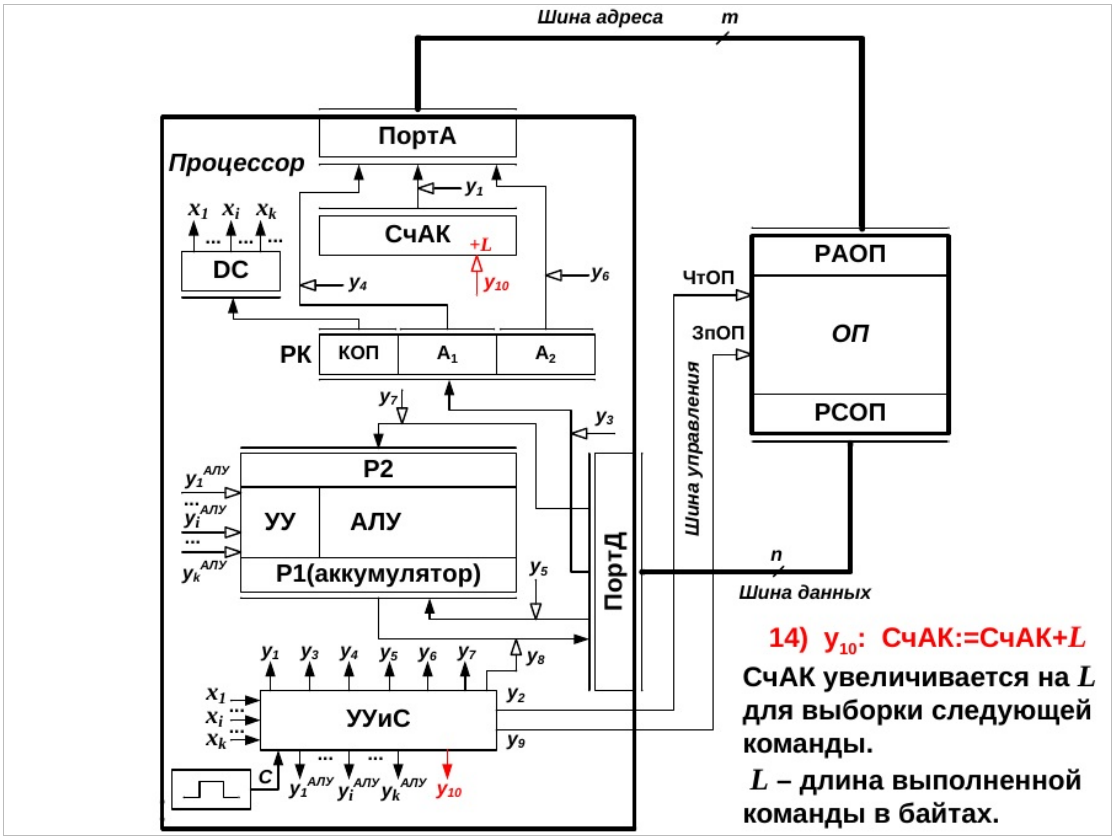
1. y8: `ПортД := Р1`
2. y4: `ПортА := А1`
3. y9: `ЗпОП`



Этап 5. Продвижение СчАК

1. y10: `СчАК := СчАК + L`, где **L** — длина выполненной команды в байтах.

Этот этап можно выполнить и сразу после выборки команды.



Микропрограмма процессора в целом

После включения УУиС бесконечно повторяет этапы цикла:

```
y1, y2, y3, y4, y2, y5, y6, y2, y7, y_i^АЛУ, y8, y4, y9, y10, …
```

Между командами выполняется микропрограмма обработки прерываний, если они возникли. При подаче сигнала Set УУиС вырабатывает y0, который устанавливает СчАК на начальный адрес программы.



1.5. Пример с конкретными числами

Дано

- 256 команд в системе команд, значит КОП = 8 бит.
- Два операнда в ОП, значит два поля адреса.
- Шина данных 32 бита: за одно обращение читается от 1 до 4 байт.
- Шина адреса 8 бит, значит ОП не больше $2^8 = 256$ байт.
- ОП хранит и данные, и команды (принстонская архитектура фон Неймана).
- В примере команда занимает 3 байта (КОП, А1, А2), операнды по 2 байта.

Пример.

Проиллюстрируем последовательность изменения содержимого регистров процессора при выполнении арифметической команды (сложение чисел с фиксированной точкой).

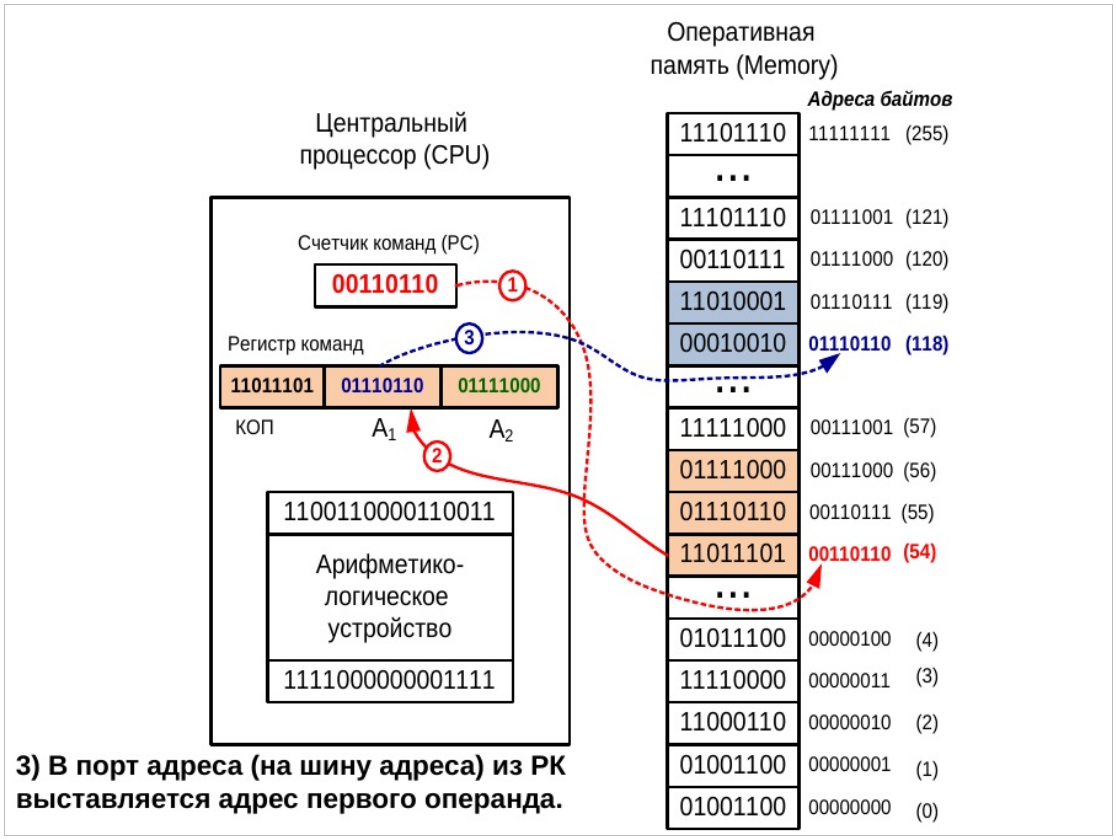
Дано:

1) **Система команд процессора включает в себя 256 команд (следовательно поле КОП должно содержать 8 бит).**
2) **В арифметической операции участвуют два операнда, находящиеся в ОП (в команде должно быть два поля для представления адресов операндов). Если длина операнда превышает 1 байт, то адресом операнда считается адрес его первого байта. Поле КОП однозначно определяет выполняемую операцию, а также тип (длину) операндов.**
3) **ОП организована как последовательность байтов, каждый байт имеет адрес (номер).**
4) **Ширина шины данных 32 бита (значит, из ОП памяти за одну операцию чтения (записи) может быть считано (записано) минимум 1 байт, максимум 4 байта).**
5) **Ширина шины адреса 8 бит (значит объем оперативной памяти не может превышать 2^8=256 байтов).**
6) **ОП хранит и данные и команды (принстонская архитектура ОП, предложенная Фон-Нейманом).**

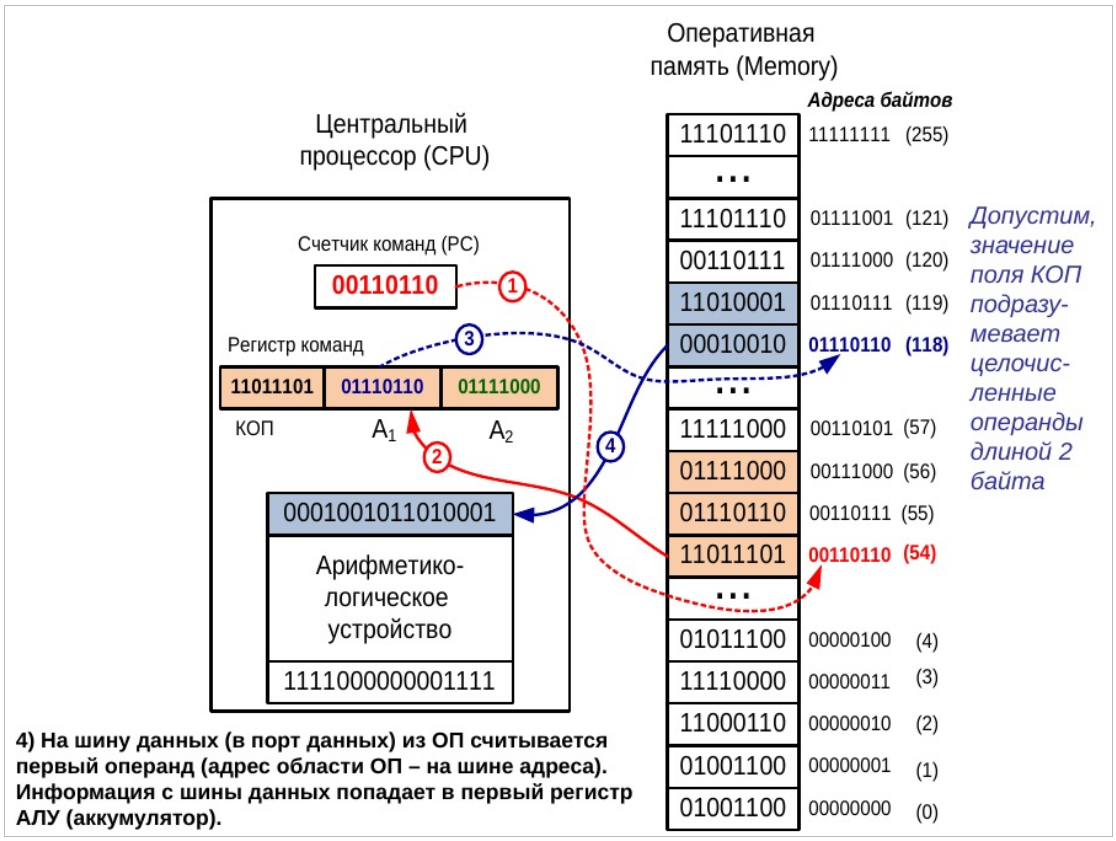
СчАК часто обозначают СК (счетчик команд) или PC (program counter).

Ход выполнения

Шаг	Что происходит
1	СК выставляется на шину адреса
2	Из ОП читается команда и попадает в РК
3	Из РК на шину адреса выставляется А1
4	Первый операнд читается в аккумулятор
5	Выставляется А2
6	Второй операнд читается во второй регистр АЛУ
7	АЛУ выполняет операцию (по КОП), результат в аккумуляторе
8	Снова выставляется А1
9	Результат записывается в ОП на место первого операнда
10	СК увеличивается на длину команды (здесь на 3)
11	Всё повторяется для следующей команды

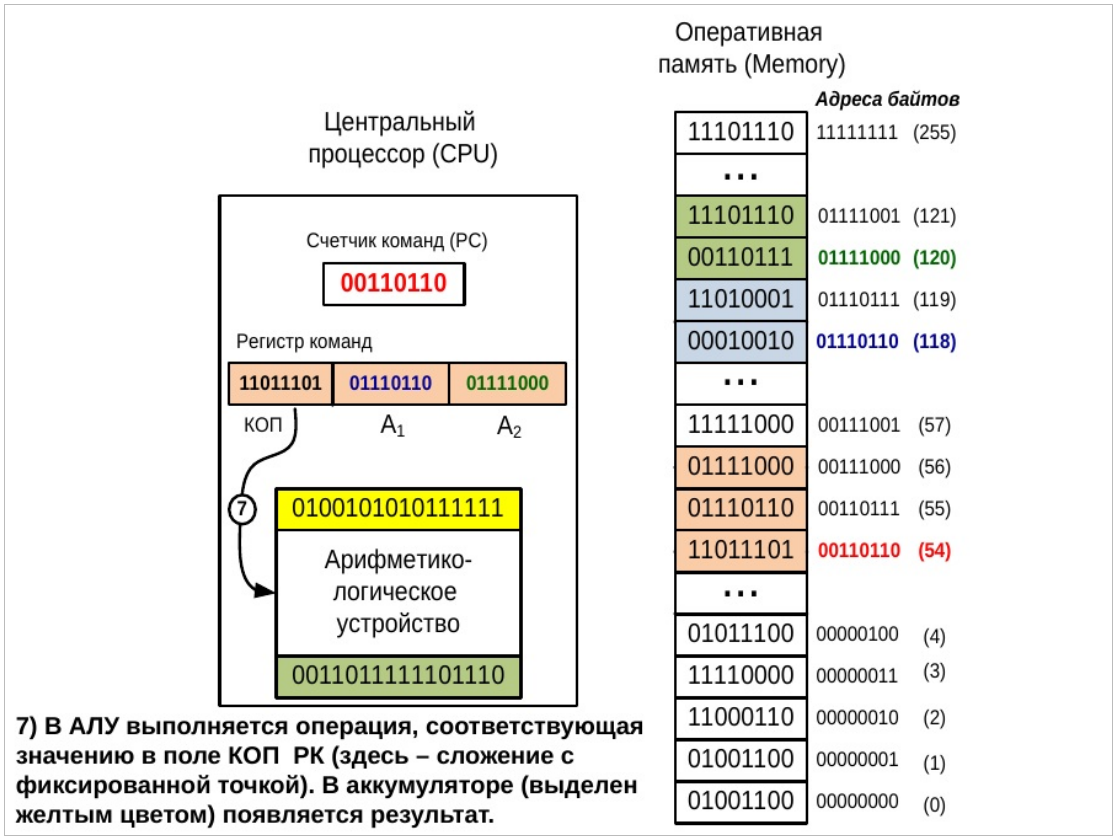


3) В порт адреса (на шину адреса) из РК выставляется адрес первого операнда.

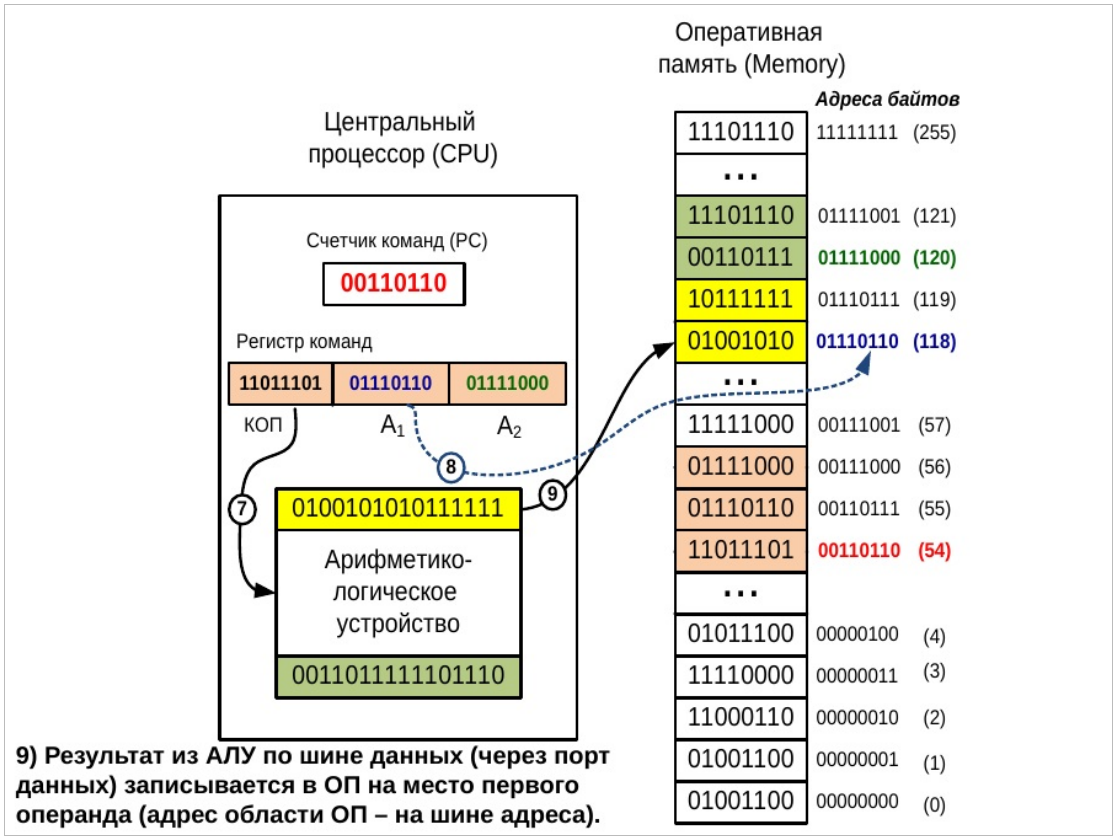


Допустим, значение поля КОП подразумевает целочисленные операнды длиной 2 байта

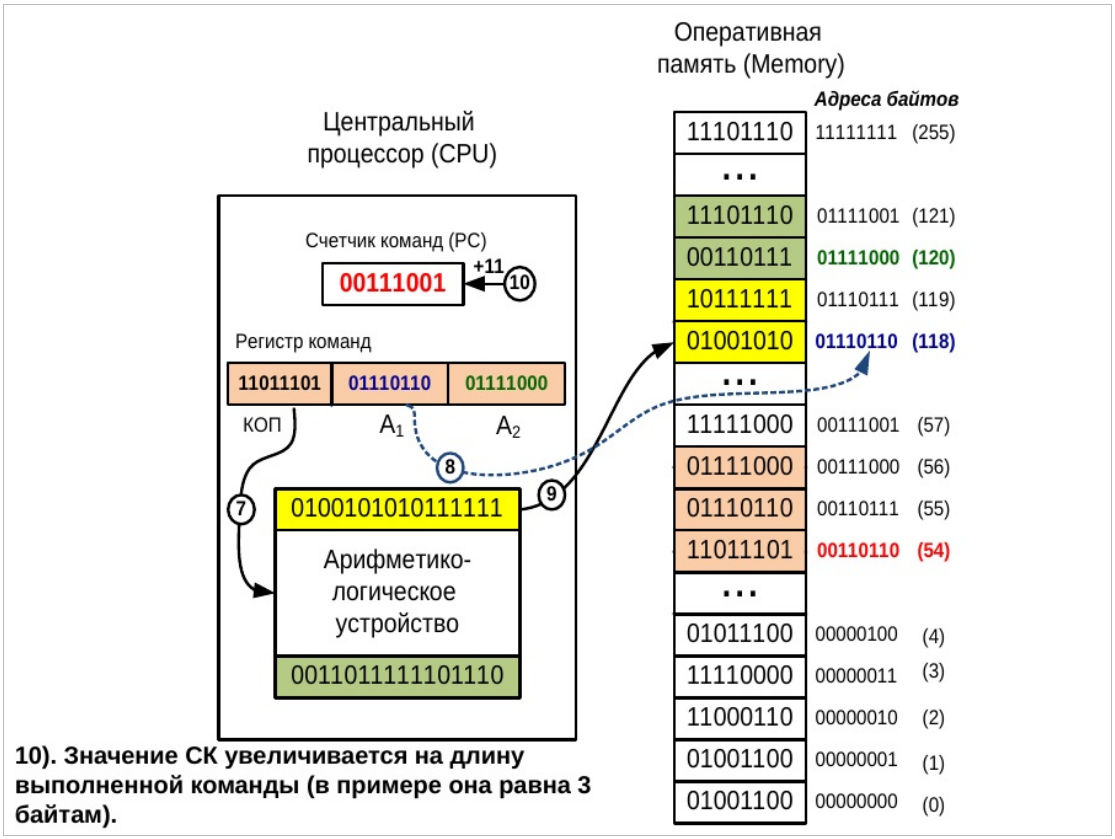
4) На шину данных (в порт данных) из ОП считывается первый операнд (адрес области ОП – на шине адреса). Информация с шины данных попадает в первый регистр АЛУ (аккумулятор).



7) В АЛУ выполняется операция, соответствующая значению в поле КОП РК (здесь – сложение с фиксированной точкой). В аккумуляторе (выделен желтым цветом) появляется результат.



9) Результат из АЛУ по шине данных (через порт данных) записывается в ОП на место первого операнда (адрес области ОП – на шине адреса).



10). Значение СК увеличивается на длину выполненной команды (в примере она равна 3 байтам).

1.6. Признак результата и флаги

Как процессор узнаёт, что АЛУ закончило? АЛУ выдаёт сигнал «занято/свободно» **Z_АЛУ = ШАЛУ(8)**: 1 значит занято, 0 значит операция выполнена, результат в Р1.

1. Уточним 3 этап рабочего цикла процессора – «Выполнение операции в АЛУ». После того, как процессор загрузит в регистры АЛУ операнды (2 этап рабочего цикла) и инициирует выполнение нужной микропрограммы в АЛУ (3 этап рабочего цикла), АЛУ устанавливает на своем выходе $Z_{АЛУ}$ (ШАЛУ(8) – сигнал «занято/свободно») единицу (занято!). Процессор ожидает, пока АЛУ не установит ноль на выходе. Это будет означать, что микропрограмма исполнения операции выполнилась, в Р1 находится результат, и теперь АЛУ готово для исполнения микропрограммы следующей операции (свободно!).

Помимо результата выполнения операции (в Р1) АЛУ формирует на выходной шине (ШАЛУ) унитарный код признака результата (только для операций сложения, вычитания, инкрементирования, декрементирования, и поразрядных логических операций: И, ИЛИ, Исключающее ИЛИ):

ШАЛУ(0)=1, если Р1=0;
ШАЛУ(1)=1, если Р1<0;
ШАЛУ(2)=1, если Р1>0;
ШАЛУ(3)=1, если произошла исключительная ситуация «Переполнение с фиксированной точкой».

Четырехразрядный унитарный код признака результата преобразуется с помощью шифратора (Coder) в двухразрядный позиционный код признака результата: (00 – результат равен нулю, 01 – результат меньше нуля, 10 – результат больше нуля, 11 – переполнение с фиксированной точкой).

Признак результата формируется только для сложения, вычитания, инкремента, декремента и поразрядных И, ИЛИ, исключающего ИЛИ. Это унитарный код на ШАЛУ(0:3):

Линия	Условие
ШАЛУ(0) = 1	P1 = 0
ШАЛУ(1) = 1	P1 < 0
ШАЛУ(2) = 1	P1 > 0
ШАЛУ(3) = 1	переполнение с фиксированной точкой

Шифратор **Coder** преобразует его в 2-разрядный позиционный код (**РПР**):

РПР	Смысл
00	результат равен нулю
01	результат меньше нуля
10	результат больше нуля
11	переполнение с фиксированной точкой

Флаги исключительных ситуаций (регистр флагов **РФ**):

- ШАЛУ(4): переполнение порядка;
- ШАЛУ(5): исчезновение порядка;
- ШАЛУ(6): потеря значимости;
- деление на ноль: на слайде указано ШАЛУ(8), но эта линия уже занята сигналом «занято/свободно», а в диапазоне ШАЛУ(3:7) пропущена 7. Скорее всего, опечатка и имелась в виду ШАЛУ(7).

Если ШАЛУ(3:7) = 00000, исключительных ситуаций нет.

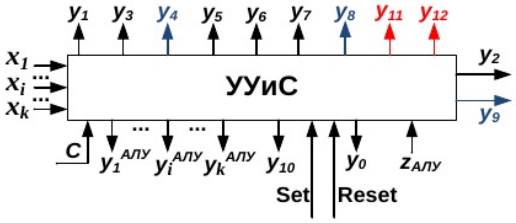
Уточнённый этап 4:

- 11.1) y8: `ПортД := P1`
- 11.2) y11: `РПР(0:1) := Coder(ШАЛУ(0:3))` (только для add, sub, inc, dec, and, or, xor)
- 11.3) y12: `РФ(0:4) := ШАЛУ(3:7)`
- 12) y4: `ПортА := A1`
- 13) y9: `ЗпОП`

Другие команды **не должны** менять РПР. Разрядность РФ больше числа флагов АЛУ, потому что прерывания бывают и от таймеров, внешних устройств и т. п.

Уточним 4 этап рабочего цикла процессора – «Запись результата в ОП»

11.1) y_8: ПортД:=P1
11.2) y_{11}: РПР(0:1):=Coder(ШАЛУ(0:3))
(только для add, sub, inc, dec, and, or, xor)
11.3) y_{12}: РФ(0:4):=ШАЛУ(3:7)
12) y_4: ПортА:=A1
13) y_9: ЗпОП



Помимо результата выполнения операции, на этом этапе (если выполнялась команда сложения, вычитания, инкрементирования, декрементирования, поразрядного И, ИЛИ, исключающего ИЛИ) сохраняется в регистре признака результата (РПР) позиционный код признака результата, сформированный шифратором (Coder). Другие команды не должны изменять содержимое РПР!

Флаги исключительных ситуаций, которые могут возникать при выполнении арифметических операций, запоминаются на регистре флагов прерываний (РФ), для их последующего анализа и обработки системой прерываний.

Если ШАЛУ(3:7)=00000, значит исключительной ситуации при выполнении операции в АЛУ не возникло, т.е. операция была выполнена успешно.

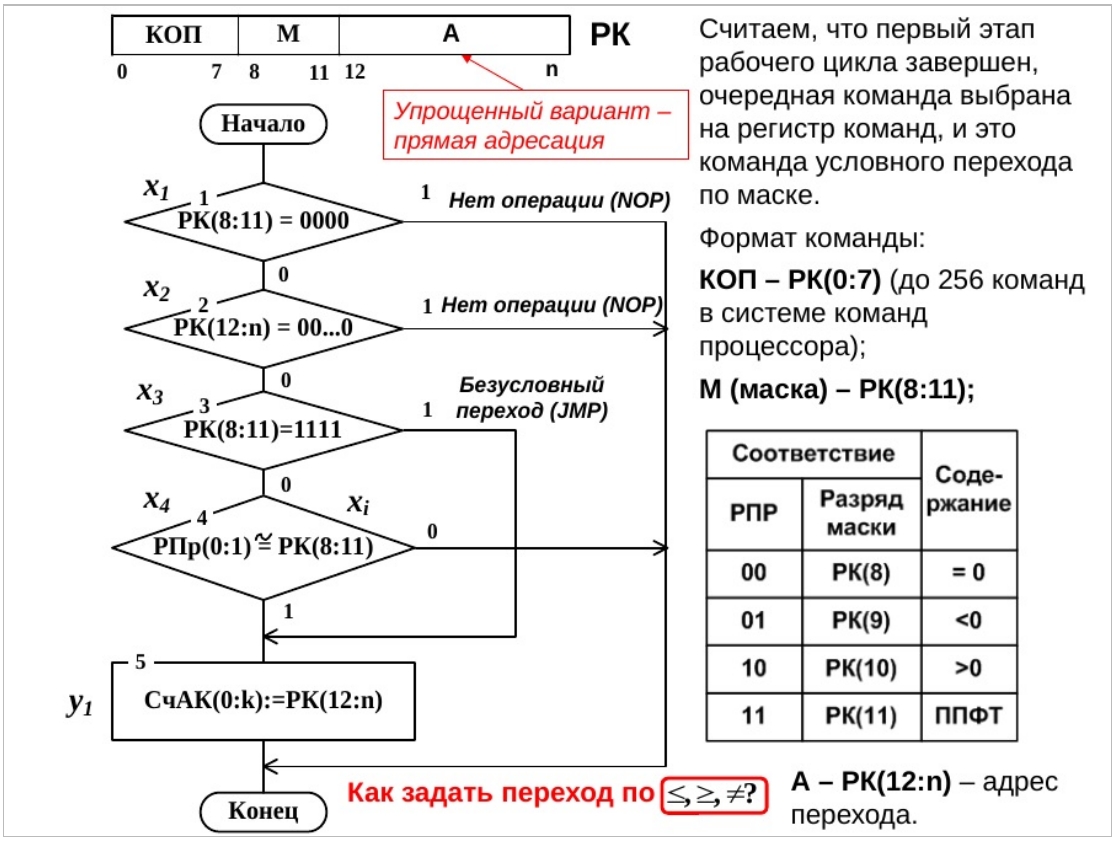
Внимание! В структуру процессора ввели еще два специальных регистра: двухразрядный РПР и (s+1)-разрядный РФ. Разрядность РФ больше числа флагов исключительных ситуаций на ШАЛУ, т.к. возможны прерывания не только от АЛУ, но и от таймеров, внешних устройств и т.п. (рассмотрим позже).

1.7. Команда условного перехода

После арифметической команды СчАК просто продвигается на длину команды. Чтобы организовать ветвление или цикл, нужны **команды перехода**. Пример: условный переход по маске (упрощённо, прямая адресация).

Формат:

- КОП: РК(0:7), до 256 команд;
- М (маска): РК(8:11);
- А (адрес перехода): РК(12:n).



Как работает

- Если в поле М или в поле А нули, переход не выполняется (команда «нет операции»).
- Если в М все единицы, переход выполняется всегда (безусловный переход).
- Если признак результата соответствует коду в М, то в СчАК (уже продвинутый после выборки) записывается адрес А. Иначе выполнение идёт по порядку.
- Признак результата не меняется. Прерываний нет.

Разрядность адреса зависит от объёма ОП (E_ОП в байтах):

```
N_A = ⌈log₂ E_ОП⌉
```

Такую разрядность имеют СчАК и поле А в РК.



1.8. Прерывания и стек

Прерывание — переключение с одной программы на другую.

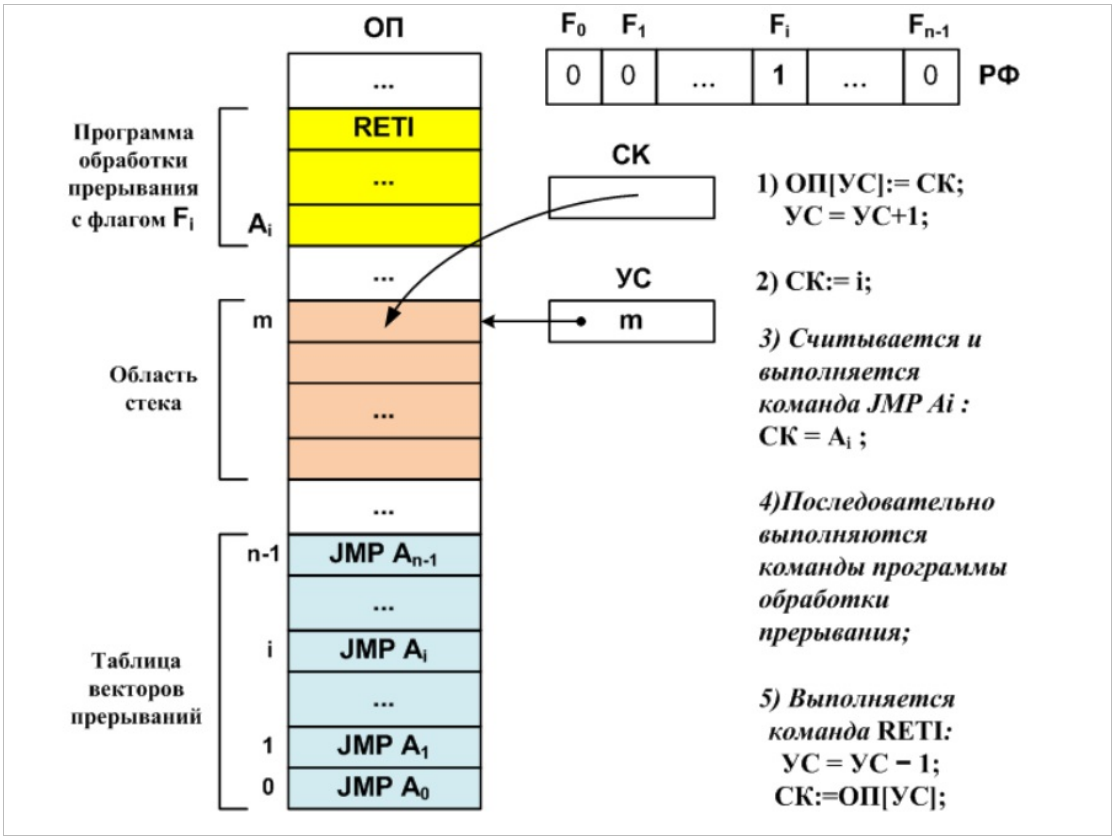
Виды:

- **Исключения** (деление на ноль, переполнение, исчезновение порядка, потеря значимости). Как правило, не маскируются.
- **От внешних устройств** (запрос ввода-вывода, нажатие клавиши Num Lock). Могут быть замаскированы.
- **От таймера.** В мультипрограммном режиме каждой программе выделяется квант времени (например, 1/30 с), по его истечении процессор переключается на другую программу.

Аппаратная поддержка

- **РФ** (регистр флагов): i-й бит соответствует i-й причине прерывания.
- **РМ** (регистр масок): единица в i-м бите означает, что это прерывание не обрабатывается.
- **УС** (указатель стека): адрес верхушки стека.
- **ТВП** (таблица векторов прерываний): первые n слов ОП (при гарвардской архитектуре — памяти программ). В i-м слове лежит команда `JMP Aᵢ` на программу обработки i-го прерывания. Программист и трансляторы эти слова не используют.

Стек — структура LIFO (последним вошёл, первым вышел). При записи информация кладётся в слово, на которое указывает УС, затем УС увеличивается. При извлечении сначала УС уменьшается, потом читается значение.



Обработка прерывания (после каждой команды процессор анализирует РФ; если $F_i = 1$):

1. Запомнить адрес возврата: `ОП[УС] = СК; УС = УС + 1`.
2. Перейти в ТВП: `СК = i`.
3. Выполнить там команду `JMP Ai`, после чего `СК = Ai`.
4. Выполнить программу обработки прерывания.
5. Последняя команда **RETI**: `УС = УС – 1; СК = ОП[УС]`. Прерванная программа продолжается.

Если в РФ несколько единиц, обработка идёт по процедуре опроса с учётом приоритетов или без них (подробно в курсе «Операционные системы»).

Недостаток векторов прерываний: запоминается только адрес возврата. Регистры специальных функций (например, РПР) и РОН, которые может испортить обработчик, должен сохранять и восстанавливать программист. Существуют системы, которые автоматически сохраняют адрес возврата и часть регистров в **слове состояния программы (ССП, PSW)**.

В ОП (если гарвардская архитектура, то в памяти программ) первые n слов отводятся под таблицу векторов прерываний (ТВП). Программист не должен их использовать при низкоуровневом программировании (на ассемблере или в машинных кодах). Трансляторы с языков высокого уровня их тоже не используют.

Каждому биту F_i в регистре флагов РФ, как уже говорилось, соответствует определенная причина прерывания и слово в ТВП.
В этом (i-ом) слове ТВП записана (в двоичном виде) команда безусловного перехода на адрес A_i первой команды программы обработки соответствующего прерывания, которая также хранится в ОП.

Для осуществления возможности возврата в определенное место прерванной программы (то, в котором она была прервана) в ОП выделяется определенная область под стек, а в структуру процессора вводится еще один специальный регистр – указатель стека (УС).

В УС всегда находится адрес слова, которое является «верхушкой» стека (верхнего свободного элемента стека).
Стек – структура данных, реализующая принцип LIFO (last in, first out) – последний вошел, первый вышел. Если информация записывается в стек, то она записывается в «верхушку стека», т.е. в слово, адрес которого записан в УС. После этого адрес в УС изменяется (в нашей иллюстрации увеличивается) так, чтобы он вновь указывал на свободное слово (верхушку стека). При извлечении информации из стека сначала изменяется значение в УС (уменьшается в нашей иллюстрации) для того, чтобы из стека было извлечено значение, записанное последним.

Часть 2. Теория: устройство процессора и системы команд

План лекции: элементы процессора; операционные устройства; устройство управления и микропрограммный автомат; процессор Intel 8086 и сегментная организация памяти; команды и их кодирование; подпрограммы.

2.1. Элементы процессора

Дешифратор команд, операционные устройства (АЛУ), устройство управления, регистры, транслятор адресов операндов, шины.

Историческая справка: в 1971 году появился первый микропроцессор **Intel 4004**. По вычислительной мощности он был равен ENIAC, стоил около 200 долларов, содержал 2300 транзисторов и работал на частоте 108 кГц.

2.2. Операционные устройства

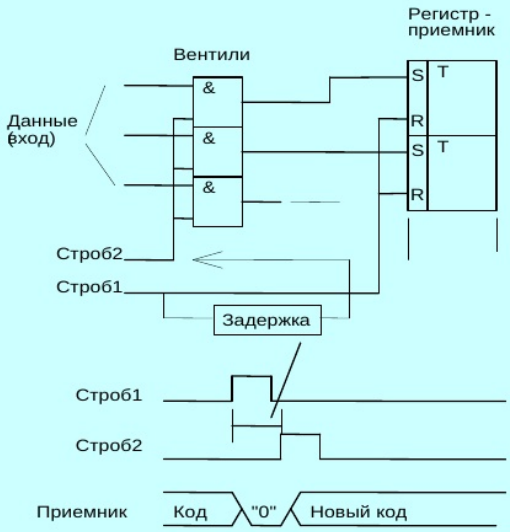
Бывают для целочисленной арифметики, логических операций, сдвигов, десятичной арифметики, чисел с плавающей запятой, графических операций.

Структурный базис — набор элементов, из которых строятся операционные устройства:

- **регистры** хранят слова данных;
- **шины** связывают регистры и передают слова данных;
- **комбинационные схемы** выполняют вычисления по управляющим сигналам от устройства управления.

Регистр — линейка триггеров. Бывают программно-видимые явно, видимые косвенно (например, «теневые» регистры дескрипторов сегментов), внутренние служебные (регистр адреса памяти, регистр данных) и внутренние для промежуточных результатов.

Простейший мультиплексор

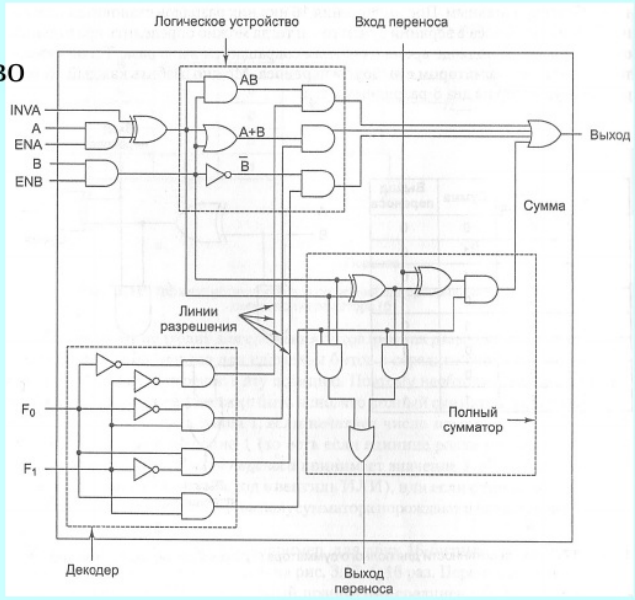


АЛУ — комбинационная схема (внутри нет элементов памяти). Принимает на два входа два операнда (например, из двух регистров) и формирует результат. Состав: логическое устройство, полный сумматор, декодер операции.

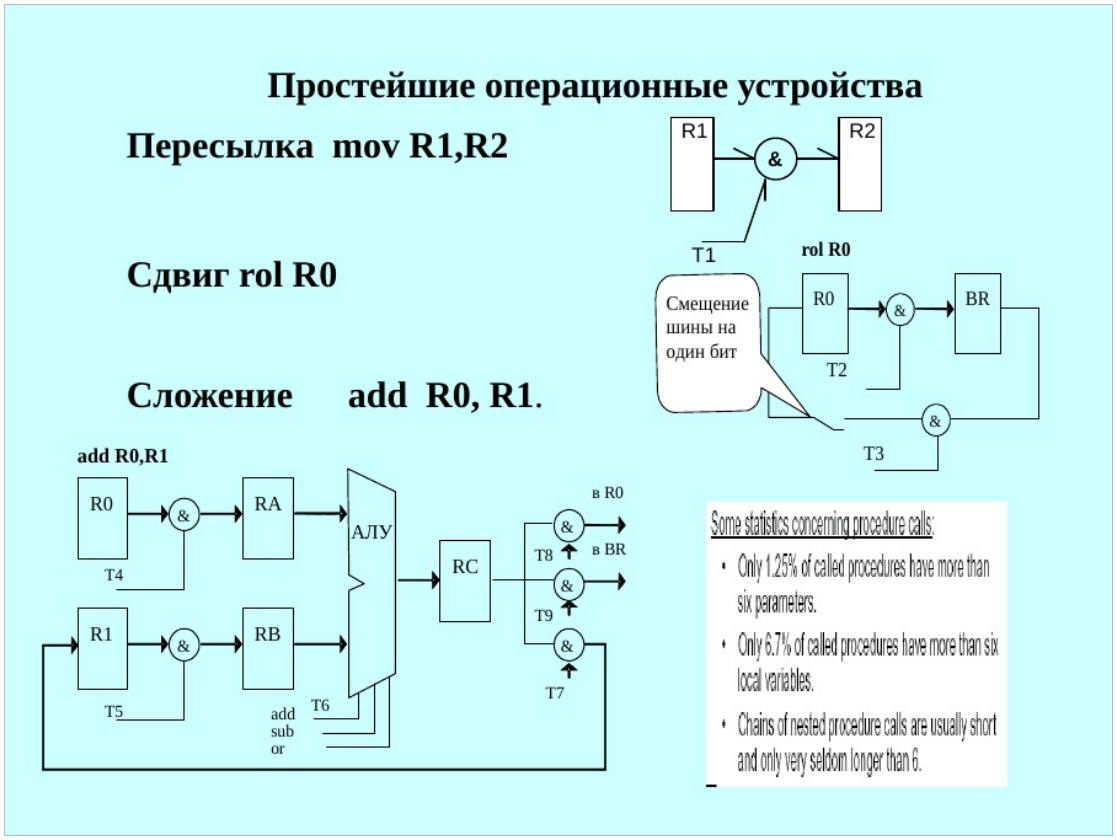
АЛУ. Принципиальная схема.

Состав

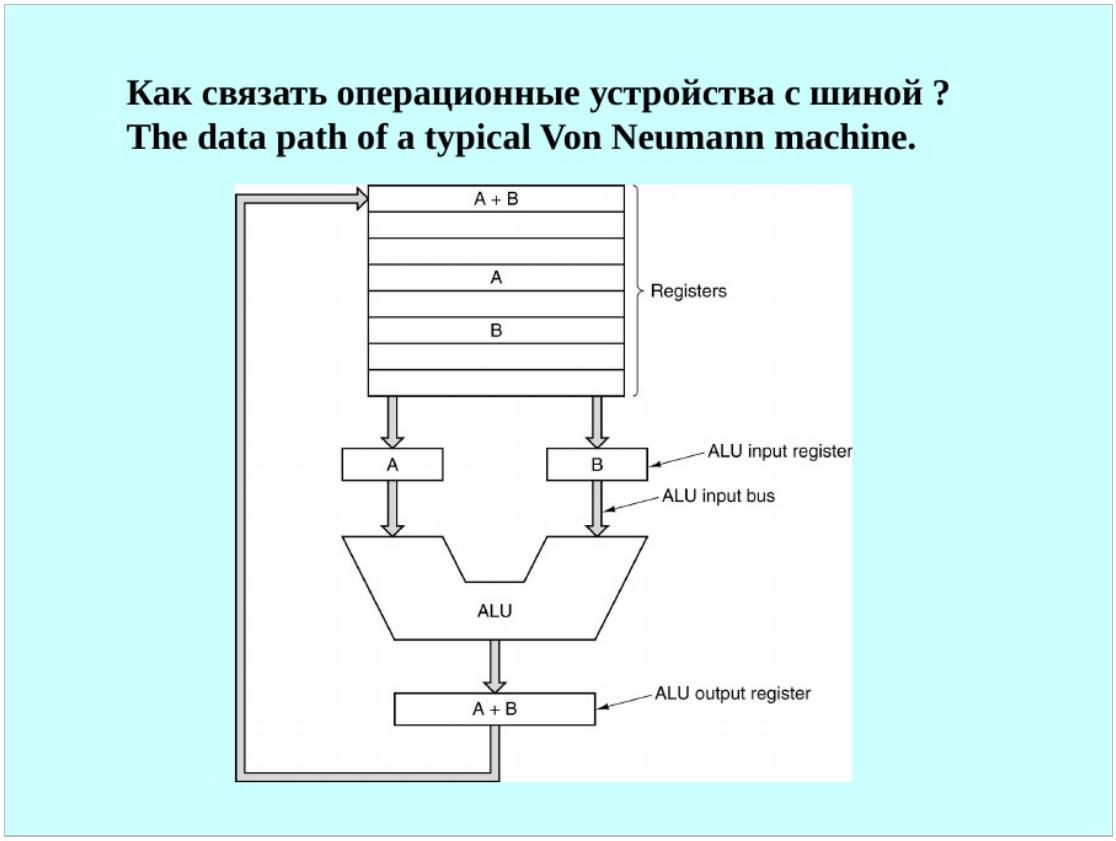
- Логическое устройство
- Полный сумматор
- Декодер операции



Простейшие операции: пересылка `mov R1,R2`, сдвиг `rol R0`, сложение `add R0,R1`.



Связь операционных устройств с шиной: пример тракта данных машины фон Неймана.



2.3. Устройство управления

Функции: управление ходом вычислительного процесса и автоматическое выполнение команд. Формирует сигналы для выборки команды из ОЗУ, дешифрации КОП, формирования адресов операндов, выборки операндов, передачи их в операционное устройство, выполнения операции, передачи результата в ОЗУ, инициирования ввода-вывода и реакции на прерывания.

Термины

- **Микрооперация** — элементарное действие за один такт синхронизации.
- **Микрокоманда** — совокупность одновременно выполняемых микроопераций.
- **Микропрограмма** — последовательность микрокоманд, определяющая машинный цикл.

Состав УУ

- Управляющая часть: регистр команды, микропрограммный автомат (+ дешифратор команд), узел прерываний и приоритетов.
- Адресная часть: операционный узел УУ, регистр адреса, счётчик команд.

Микропрограммный автомат с жёсткой логикой

- Сигналы управления реализованы однажды соединёнными логическими схемами.
- КОП определяет, какие сигналы и в какой последовательности вырабатывать.
- Сигналы привязаны к импульсам синхронизации. Счётчик тактов сбрасывается в T1 по окончании цикла команды.
- Цикл команды может занимать разное число тактов. На каждом такте вырабатывается своя микрокоманда.
- На выработку сигналов влияют также флаги.

Микропрограммный автомат с программируемой логикой

- Идею микропрограммирования предложил в 1951 г. **Морис Уилкс** (Кембридж). Между командами и сигналами управления появляется уровень микропрограмм.
- Отличительная особенность: **память микропрограмм**. Каждой команде соответствует микропрограмма.
- К 70-м годам идея стала преобладающей, но в современных процессорах от неё отказались, так как она сдерживала рост производительности.

2.4. Архитектуры системы команд

Классификация по тому, где хранятся операнды и как к ним обращаются: стековая, аккумуляторная, регистровая, с выделенным доступом к памяти.

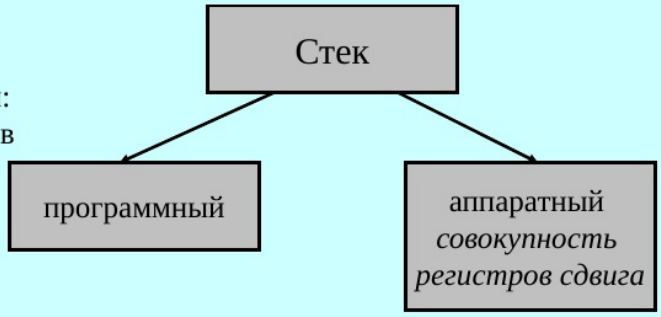
Архитектура	Суть	Плюсы и минусы
Стековая	Операнды в стеке (LIFO), операции push и pop, запись только в верхнюю ячейку	Меньше издержек на адресацию. Выражения записываются в обратной польской нотации
Аккумуляторная	Один операнд и результат в аккумуляторе, второй читается из памяти. Команды `load x`, `store x`	Короткие команды, простое декодирование. Но один регистр даёт много обращений к памяти
Регистровая	Массив регистров общего назначения (РОН), к любому обращаются по номеру. Подвиды: регистр-регистр, регистр-память, память-память	Компактный код, высокая скорость. Команды длиннее, чем в аккумуляторной
С выделенным доступом к памяти	К памяти обращаются только `load` и `store`, операнды в обработке только в регистрах	Характерна для RISC (команды обычно 32 бита, трёхадресные). Примеры: SPARC, MIPS R10000, DEC Alpha, PowerPC

В CISC число РОН обычно невелико, в RISC их до нескольких сотен.

Стековая архитектура

Стек - память, состоящая из взаимосвязанных ячеек, взаимодействующих по принпипу “последним вошел, первым вышел” (LIFO, Last In First Out).

Для работы со стеком предусмотрены две операции: push (проталкивание данных в стек) и pop (выталкивание данных из стека). Запись возможна только в верхнюю ячейку стека.

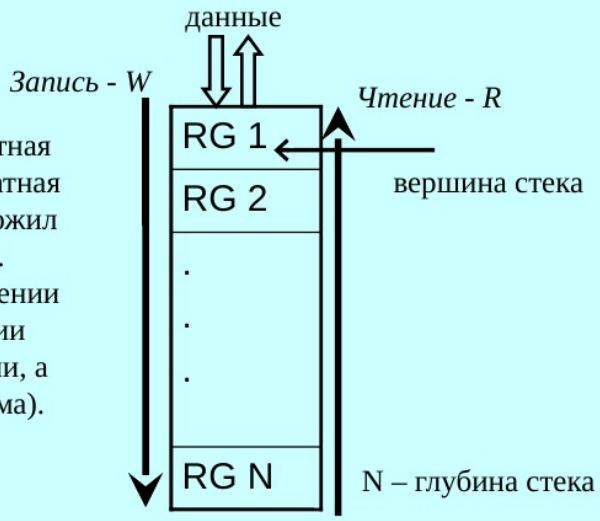


Механизм стеков снижает издержки на адресацию.
При стековой организации все данные идентифицируются вершиной стека.

Обратная польская запись (Я. Лукашевич): скобок нет, знак операции стоит после операндов. Выражение `a = a + b + a × c` записывается как `a = a b + a c × +`.

Стековая архитектура

При описании вычислений с использованием стека обычно используется иная форма записи математических выражений, известная как обратная польская запись (обратная польская нотация), которую предложил польский математик Я. Лукашевич. Особенность ее в том, что в выражении отсутствуют скобки, а знак операции располагается не между операндами, а следует за ними (постфиксная форма). Последовательность операций определяется их приоритетами. Выражение *a =a+b + a x c* в постфиксной форме будет записано в виде: *a = a b + a c x+.*



2.5. Процессор i8086 и его команды

Регистр флагов 8086

Флаг	Название
CF	переноса
PF	чётности
AF	вспомогательного переноса
ZF	нуля
SF	знака
TF	прерывания для отладки (trap)
IF	разрешения прерывания
DF	направления цепочечных команд
OF	переполнения

Сегментация памяти. Регистры сегментов: `cs` (код), `ds` (данные), `ss` (стек), `es` (дополнительный). Полный адрес записывается как `ds:bx`, например `0xb800:0`. Для работы с адресами: `lea bx, iA` и `les bx, pA`. Модели памяти: Tiny, Small, Medium, Compact, Large, Huge.

Особенности набора команд

- Всего 538 команд: целочисленные 210, с плавающей точкой 85, MMX 57, MMX Pentium III 16, XMM 70.
- x86 относится к классу **CISC**.
- Базовая структура двухоперандная: один операнд всегда в регистре, второй в регистре или памяти.
- Длина команды от 1 до 15 байт, кодирование сложное, адресация многокомпонентная.
- Система команд не ортогональна: многие команды и регистры специализированы.
- Команды пересылки не меняют признаки.

Группы команд

1. Пересылки

- `MOV eax,[ebx]`: общая пересылка.
- `PUSH` / `POP`: стек (указатель (E)SP показывает на последний занятый элемент).
- `LDS, LES, LFS, LGS, LSS` загружают «длинные адреса», `LEA` загружает эффективный адрес.
- Строковые: `MOVSB/W/D`, `LODSB/W/D`, `STOSB/W/D`. С префиксом `rep` повторяются, пока `[e]cx` не станет 0. Адреса меняются на ±1 в зависимости от DF.
- `CMOVcc`: условная пересылка (с Pentium Pro), позволяет обойтись без условных переходов.
- `BSWAP`, `XCHG`: обмен.
- `IN` / `OUT`: обмен с портами периферии (номер порта 0…255 в команде либо в `DX`).

2. Обработка

- Арифметические: `ADD, ADC, SUB, SBB, CMP, NEG, INC, DEC, MUL, DIV, IMUL, IDIV, SAR/SAL`. Для `MUL/DIV` разрядность приёмника произведения и источника делимого вдвое больше, чем у сомножителей, частного и остатка.
- Логические: `OR` (установка битов), `AND` (сброс битов), `XOR` (выборочное инвертирование), `NOT`, `TEST`, `BSF/BSR` (поиск единичного бита), `BT/BTC/BTR/BTS` (проверка бита).
- Сдвиги: циклические `ROR, ROL, RCR, RCL` (RC* через флаг CF), арифметические `SAR, SAL` (знаковый бит сохраняется), логические `SHL, SHR` (освободившиеся позиции заполняются нулями), `SHLD, SHRD` (двойное слово).



3. Проверки и передача управления

- `CMP`, `CMPS`, `SCAS`: сравнение с установкой флагов.
- `Jcc`: условные переходы (в x86 их 63). Для сравнения чисел со знаком используются сокращения от greater и less.
- `LOOP`: цикл со счётчиком в `[e]cx` (вложенные циклы организовать затруднительно).
- `JMP`: безусловный переход.
- `CALL`, `RET`, `ENTER`, `LEAVE`: поддержка модульной структуры.

Формат команды

```
[Префикс] КОП [постбайт адресации] [смещение] [непосредственный операнд]
```

Пример: `add [eax+2*ebx], ecx` кодируется как `000000 01 | 00 001 100 | 01 011 000`.

Память DOS

- **EMS**: расширенная (expanded) память.
- **XMS**: стандарт управления памятью за барьером 640 КБ (драйвер HIMEM.SYS).
- **UMB**: блоки верхней памяти.
- **Shadow RAM**: теневая память, перезапись данных из медленного ПЗУ в быстрое ОЗУ.

2.6. Подпрограммы

Два подхода к структурированию программы:

- **макроподстановки**: текст фрагмента подставляется на этапе компиляции;
- **подпрограммы**: связь по управлению и данным, запоминание контекста, динамическое распределение памяти.

Контекст программы — всё, что нужно восстановить, чтобы продолжить выполнение: счётчик команд, регистр флагов, остальные регистры, указатели на стеки ядра и пользователя, указатели на адресное пространство задачи (CR3 в x86), промежуточные результаты.

Как облегчают сохранение контекста

1. Специальные команды: `pusha` и `popa` сохраняют и восстанавливают регистры в порядке (e)ax, (e)cx,

(e)dx, (e)bx, (e)sp, (e)bp, (e)si, (e)di.

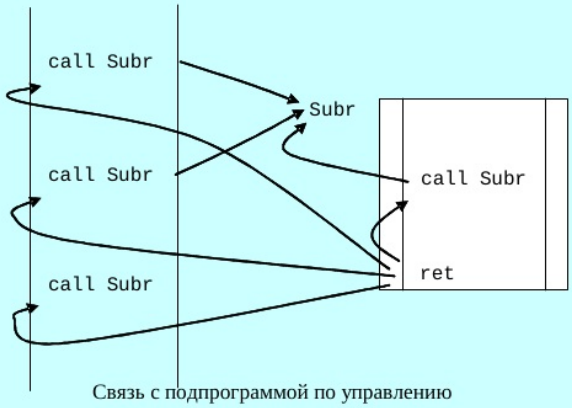
2. Несколько наборов регистров: TMS9900 (регистры в памяти, адрес задаёт указатель рабочей области), Transputer (3 регистра в виде стека), IA-64 Itanium (динамическое переименование регистров при вызове).

Вызов и возврат. `call` (в других процессорах `jsr`) автоматически запоминает адрес возврата, то есть адрес следующей команды. Минимальные действия: сохранить адрес возврата и загрузить в счётчик команд адрес первой команды подпрограммы. `ret` (`rts`) возвращает управление.

Обращение к подпрограммам - передача управления

Команда обращения к подпрограмме ОП (для этой команды используются мнемоники **call, jsr**). Эта команда должна автоматически запоминать адрес возврата (т.е. адрес команды, следующей за командой **call**).

Команда возврата из ПП (используются мнемоники **ret**, **rts**).

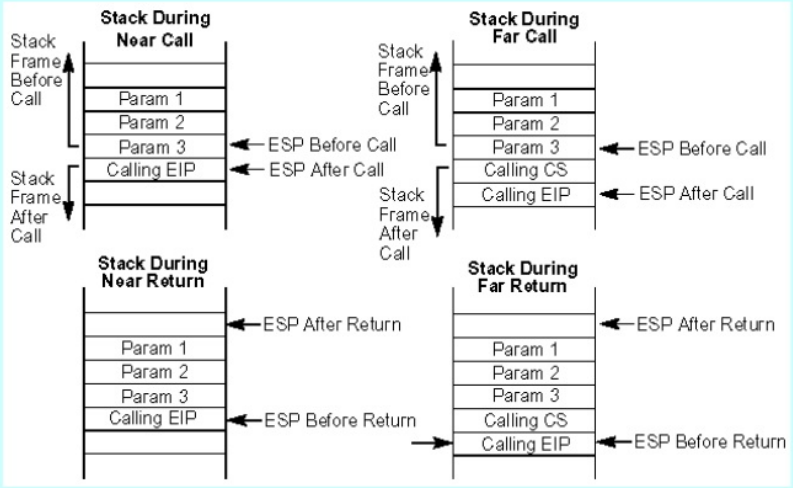


Связь с подпрограммой по управлению

Обращение к подпрограммам - сохранение/восстановление контекста

Минимальный набор действий, которые выполняет такая команда call:

- сохранение адреса возврата (т.е. адреса команды, следующей за **call**)
- загрузка в счетчик команд адреса первой исполняемой команды ПП.



Передача данных в подпрограмму: сами данные или адрес участка памяти. Они могут лежать в

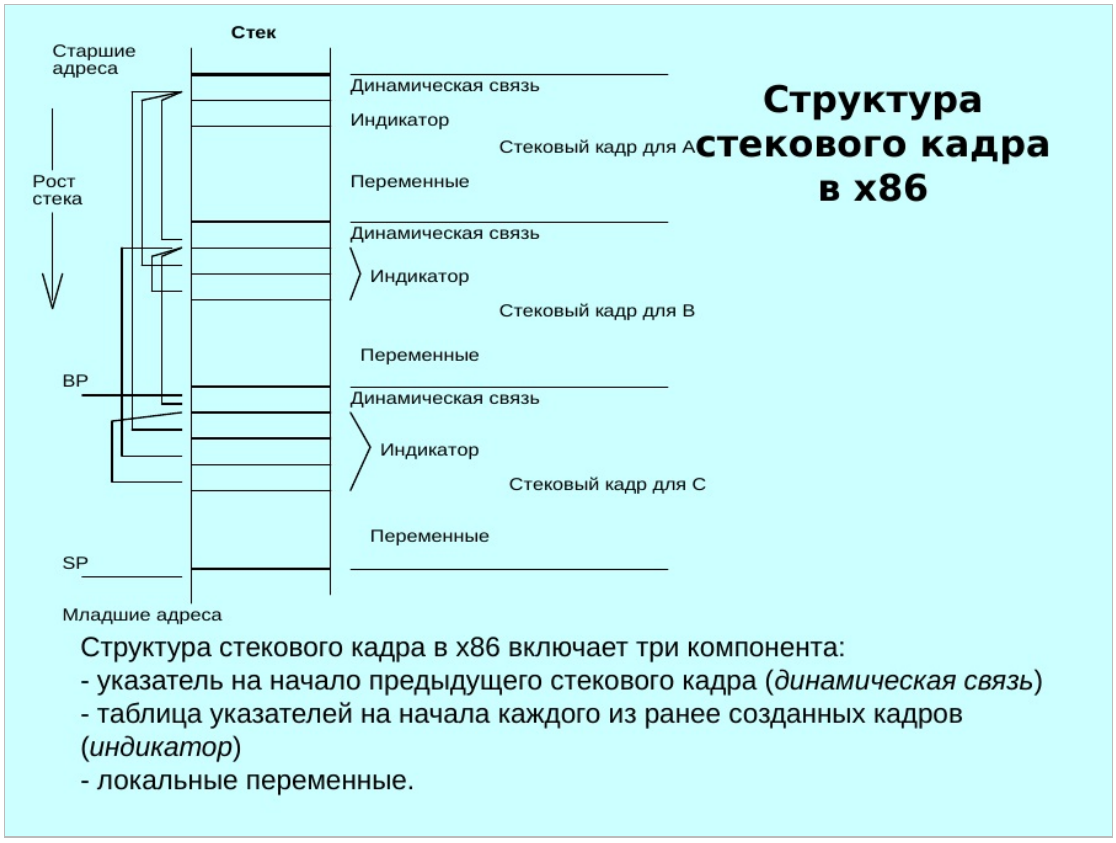
регистрах (если данных мало или регистров много), в стеке или в известном месте памяти (например, в DEC16 это слова после команды вызова).

Пример на Си: `sum = addition(a, b);`, где `addition(x, y)` считает `iX = 2*x + y`, `iY = 3*y - x` и возвращает `iX*iY`. Дизассемблирование показывает работу со стеком в отладчике Turbo Debugger.

Стековый кадр в x86 состоит из трёх частей:

- указатель на начало предыдущего кадра (динамическая связь);
- таблица указателей на начала ранее созданных кадров (индикатор);
- локальные переменные.

Для доступа к кадру используется регистр `BP`.

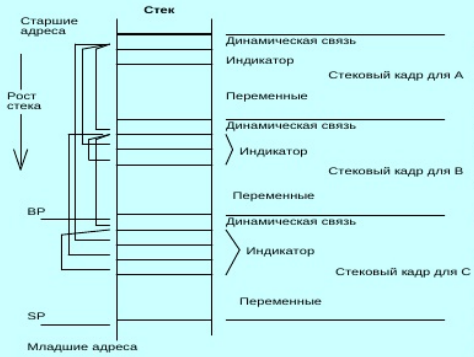


Команда `enter frmsiz, level` (frmsiz — байты под локальные переменные, level — уровень вложенности; для внешнего уровня level = 1):

1. `TmpREG = SP`: запомнить адрес начала следующего кадра.
2. `BP` в стек: динамическая связь.
3. Повторить (level − 1) раз: `BP -= 2`, `[BP]` в стек (копирование индикатора предыдущего кадра).
4. `TmpREG` в стек: ссылка на текущий кадр в индикаторе.
5. `BP = TmpREG`.
6. `SP -= frmsiz`: выделить место под локальные переменные.

`leave` освобождает стековый кадр.

Структура стекового кадра в x86. Команды ENTER и LEAVE



enter frmsiz, level

Первый параметр команды определяет количество байтов, требуемых в стеке для временных (локальных) переменных. Второй параметр показывает уровень "рекурсивности" (для самого внешнего уровня level=1). Для доступа к элементам стекового кадра используется адресный регистр BP, который указывает на начало (самый старший адрес) стекового кадра.

Выполнение команды ENTER:

1.TmpREG = SP ; Запомнить адрес начала следующего стекового кадра
2.BP -> в стек ; Создание динамической связи
3.Повторить (level-1) раз: ; Создание индикатора
BP -=2
[BP] -> в стек ; Копирование элементов индикатора из предыдущего кадра
4.TmpREG -> в стек ; Добавление в индикатор ссылки на данный кадр
5.BP = TmpREG ; Установить BP на начало создаваемого кадра
6.SP -= frmsiz ; Выделение в стеке блока под временные переменные